

CATDOKU UNITY PROJECT

Meowdoku (Catdoku)

A color-region puzzle game where players place cats (queens) on a grid, inspired by the N-Queens problem. A Unity 2D mobile project (Android/iOS) developed by **FoxGames**.

Property	Value
Unity Editor	6000.3.16f1 (Unity 6)
Render Pipeline	Universal Render Pipeline (URP)
Product Name	Catdoku
Main Scene	Assets/Scenes/Game.unity
Level Count	~1,500 JSON files (level_1 → level_999 + Tutorial.json)

Gameplay Overview

Players place **cats (queens)** on cells within a color-region grid. Each row, column, and color region must contain exactly one cat. Some cats are pre-revealed (catRevealed). Players can mark cells with X marks, use hint boosters, and have up to **3 lives** per level.

Directory Structure

```
Meowdoku/
├── Assets/                                # All game content
│   ├── Scenes/                           # Unity scenes
│   ├── Scripts/                          # Main C# source code
│   ├── Resources/                        # Runtime assets loaded via Resources.Load
│   ├── Materials/                        # Render materials
│   ├── Plugins/                          # Third-party libraries (DOTween, Android/iOS)
│   ├── GoogleMobileAds/                  # AdMob SDK
│   ├── ExternalDependencyManager/        # Android/iOS dependency management
│   └── Settings/                          # URP / build scene configuration
└── Packages/                             # Unity Package Manager (manifest.json)
```

```
|— ProjectSettings/          # Unity project settings
|— UserSettings/            # Personal editor settings (do not commit)
```

Directories to Ignore When Reading Code

Directory	Description
<code>Library/</code>	Unity build cache, auto-generated
<code>Temp/</code>	Temporary build/compile files
<code>Logs/</code>	Editor logs

Assets/Scripts — Source Architecture

Core Gameplay

File	Role
<code>GameManager.cs</code>	Central orchestrator: level lifecycle, score, lives, win/lose, boosters
<code>BoardManager.cs</code>	Loads levels, manages the board, handles cell placement events
<code>CellView.cs</code>	Cell: input (click, swipe), state (X, Queen), animations
<code>HintBooster.cs</code>	Hint booster (via <code>hintPlan</code> or algorithm)
<code>FindCatQBooster.cs</code>	Booster that finds a missing cat position
<code>BoosterChargeStore.cs</code>	Persists booster usage counts

Data (`Scripts/Data/`)

File	Role
<code>LevelData.cs</code>	Level data model: <code>solution</code> , <code>colorMap</code> , <code>catRevealed</code> , <code>hintPlan</code>
<code>LevelLoader.cs</code>	Reads/writes level JSON from <code>Resources/Levels</code>
<code>LevelValidator.cs</code>	Validates level correctness
<code>LevelCatSuggester.cs</code>	Suggests cat positions during level design
<code>LevelColorConfig.cs</code>	ScriptableObject for color palette configuration
<code>LevelPalette.cs</code>	Color palette per region

UI (`Scripts/UI/` — namespace `ColorCubeShooter`)

File	Role
<code>UIManager.cs</code>	Navigates between panels
<code>HomePanel.cs</code>	Home screen
<code>GamePanel.cs</code>	In-game HUD (score, lives, boosters)
<code>WinPanel.cs</code> / <code>LosePanel.cs</code>	Result screens
<code>TutorialPanel.cs</code>	Tutorial UI
<code>SettingPopup.cs</code>	Settings (sound, music)
<code>BoardView.cs</code>	Renders the board from <code>LevelData</code>
<code>BoosterButtonView.cs</code>	Booster buttons on the HUD
<code>BasePanel.cs</code> / <code>BasePopup.cs</code>	Base classes for panels/popups
<code>SafeArea.cs</code>	Mobile safe area handling

Tutorial

File	Role
<code>TutorialManager.cs</code>	Manages step-by-step tutorial flow
<code>TutorialConfig.cs</code>	ScriptableObject for tutorial step configuration
<code>TutorialHandAnimator.cs</code>	Tutorial hand pointer animation
<code>TutorialProgressStore.cs</code>	Persists tutorial progress (<code>PlayerPrefs</code>)

Progress & Audio

File	Role
<code>CampaignProgressStore.cs</code>	Persists current campaign level
<code>SoundManager.cs</code>	Plays background music and sound effects
<code>SoundConfig.cs</code>	ScriptableObject listing audio clips

Ads

File	Role
<code>AdsControl.cs</code>	AdMob + Unity Ads integration (enabled via <code>ADS_ENABLED</code> symbol)

File	Role
<code>RewardAdsService.cs</code>	Rewarded ad service

Editor (`Scripts/Editor/`)

File	Role
<code>LevelEditorWindow.cs</code>	Level create/edit/view tool in Unity Editor
<code>LevelColorConfigUtility.cs</code>	Utility for editing <code>LevelColorConfig</code>
<code>CellPrefabBuilder.cs</code>	Builds the cell prefab

Assets/Resources — Runtime Data

```
Resources/
├─ Levels/           # ~1,500 JSON level files
├─ Config/           # ScriptableObject: colors, sound, tutorial
├─ GameObjects/      # Prefabs (Cell.prefab)
├─ Images/           # UI sprites by screen
│   ├─ MainMenu/
│   ├─ GameView/
│   ├─ GameEnd/
│   ├─ Setting/
│   ├─ Guide/
│   ├─ Splash/
│   ├─ Icon/
│   ├─ DailyChallenge/
│   ├─ Travel/
│   └─ StartLevelHards/
├─ Sounds/           # .ogg files (music, SFX)
└─ Font/             # Fonts
```

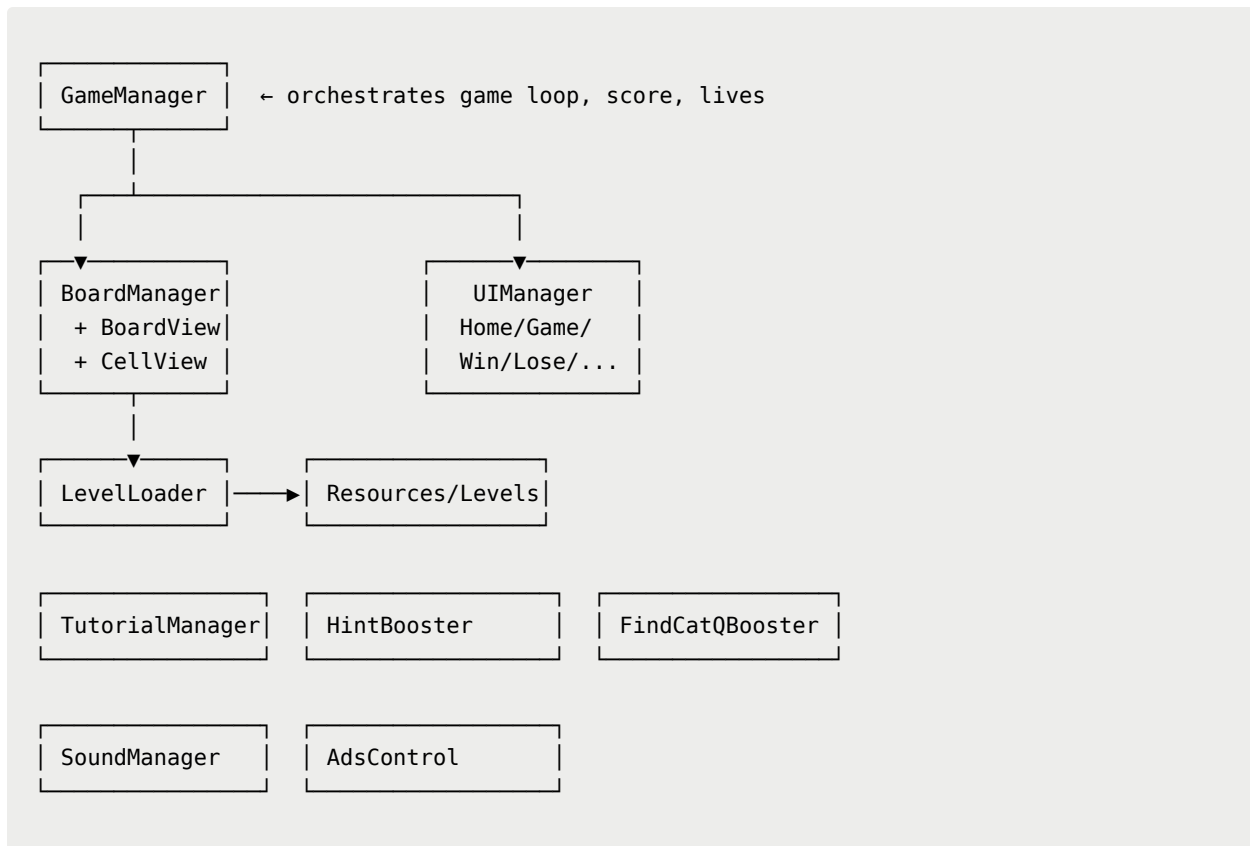
Level JSON Format

```
{
  "solution": [ [".", ".", "Q", "."], ... ],
  "colorMap": [ [1, 1, 2, 1], ... ],
  "hintPlan": []
}
```

Field	Description
<code>solution</code>	Solution grid: <code>"Q"</code> = cat, <code>"."</code> = empty cell
<code>colorMap</code>	Color region ID matrix; <code>"Q"</code> = pre-revealed cat
<code>catRevealed</code>	(optional) Boolean matrix of revealed cats
<code>hintPlan</code>	(optional) Hint steps for the hint booster

Levels are named `level_{number}.json` (e.g. `level_1.json`). The tutorial level is `Tutorial.json`.

Architecture Flow



Third-Party Libraries & Plugins

Library	Location	Purpose
DOTween	<code>Assets/Plugins/Demigiant/DOTween/</code>	UI and cell animations

Library	Location	Purpose
Google Mobile Ads	<code>Assets/GoogleMobileAds/</code>	AdMob ads
Unity Ads	Package <code>com.unity.ads</code>	Unity Ads (fallback)
External Dependency Manager	<code>Assets/ExternalDependencyManager/</code>	Resolves Android/iOS dependencies

Key Unity Packages (`Packages/manifest.json`)

- `com.unity.inputsystem` — New Input System
- `com.unity.render-pipelines.universal` — URP
- `com.unity.ugui` — UI Toolkit / uGUI
- `com.unity.feature.2d` — 2D feature set
- `com.boxqkrtm.ide.cursor` — Cursor IDE support

Development Tools

Level Editor

Open in Unity Editor: **Window** → **Level Editor** (`LevelEditorWindow`).

Features:

- View / edit / create new levels
- Paint color regions, place pre-revealed cats
- Validate levels
- Save directly to `Assets/Resources/Levels/`

Test Mode

In `GameManager`, enable `_testMode` to run a fixed level (`_levelTestIndex`) without affecting campaign progress.

Debug Hotkeys (while playing)

Key	Action
<code>H</code>	Use Hint Booster

Key	Action
F	Use Find Cat Booster

Build

The project supports **Android** builds (Gradle/IL2CPP configuration in `Library/Bee/Android/`). The `ADS_ENABLED` scripting define symbol controls whether ad modules are compiled in.

Player Data Persistence

All progress is saved via `PlayerPrefs`:

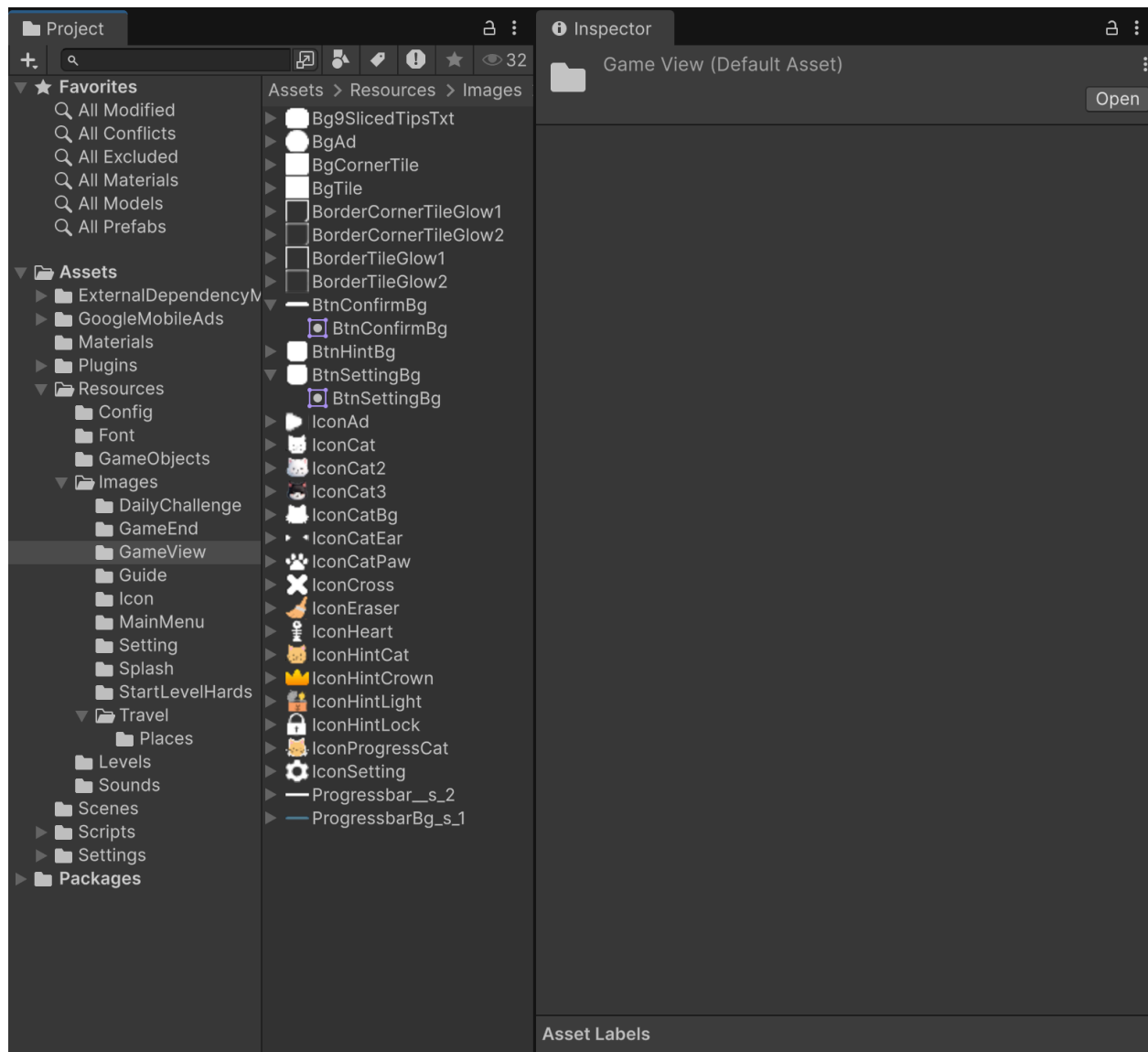
Store	Content
<code>CampaignProgressStore</code>	Current campaign level
<code>TutorialProgressStore</code>	Completed tutorial steps
<code>BoosterChargeStore</code>	Booster usage counts

Quick Start

1. Open the project in **Unity 6000.3.16f1** (or a compatible Unity 6 version).
2. Open the scene `Assets/Scenes/Game.unity`.
3. Press **Play** to run the game.
4. Use the **Level Editor** to edit or create new levels.

How to reskin project

Go to **Assets** → Images folder → **Sprites**



Monetization – Unity Ads & AdMob

This project comes pre-integrated with both **AdMob** and **Unity Ads**. You can choose

the ad network you'd like to use by entering your keys in the **AdsController** GameObject.

Locate the AdsController

- In the **Hierarchy**, go to:

Game > GameManager > AdsController

- In the **Inspector**, you'll find a component named **AdsControl (Script)** with multiple ad key fields.

